Reviewer Pack — Tropical Variety Dimension Gap in Read-Twice vs Read-Once BP…

Entry d35dbb9412cf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 17:56:58 UTC

Tropical Variety Dimension Gap in Read-Twice vs Read-Once BPs

Entry ID: `d35dbb9412cf` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-12 17:56:58 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Read-Twice Branching Programs

Statement:

> For a read-twice branching program P over n variables, the dimension of its tropicalization (as a tropical variety) satisfies dim(trop(P)) ≥ n/2. For read-once BPs, dim(trop(P)) ≤ log n + 1.

Rationale (proposer's reasoning):

> Tropical geometry captures combinatorial dependencies in BPs via tropical varieties. Read-twice BPs introduce multiplicative interactions that expand tropical dimension linearly, while read-once BPs have additive structure limiting dimension to logarithmic growth.

Taxonomy category: `LIFTING` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `96cba4e74aad3d6f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below the pivot
        factor = Fraction(1, matrix[i][i])
        for j in range(i+1, n):
            matrix[j][i] *= factor

        # Eliminate above the pivot
        for j in range(i):
            factor = matrix[j][i]
            for k in range(n):
                matrix[j][k] -= factor * matrix[i][k]
    return matrix

def tropical_dimension(matrix):
    n = len(matrix)
    rank = 0
    for i in range(n):
        if all(abs(matrix[j][i]) == float('inf') for j in range(n)):
            continue
        rank += 1
    return rank

def random_read_once_bp(n):
    bp = []
    for _ in range(2**n):
        state = [random.randint(0, n-1) for _ in range(n)]
```

```
        bp.append(state)
    return bp

def random_read_twice_bp(n):
    bp = []
    for _ in range(2**(n+1)):
        state = [random.randint(0, n-1) for _ in range(n)]
        bp.append(state)
    return bp

def tropical_matrix(bp):
    n = len(bp[0])
    m = len(bp)
    matrix = [[float('inf')] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            if bp[i][j] == 0:
                matrix[i][j] = 0
            else:
                matrix[i][j] = float('inf')
    return matrix

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 10
    read_once_bp = random_read_once_bp(n)
    read_twice_bp = random_read_twice_bp(n)

    read_once_dim = tropical_dimension(tropical_matrix(read_once_bp))
    read_twice_dim = tropical_dimension(tropical_matrix(read_twice_bp))

    result = {
        "metric_name": "tropical_dimension",
        "metric_value": read_twice_dim,
        "instances_tested": 1,
        "conjecture_holds": read_once_dim >= n/2 and read_twice_dim <= math.log(n, 2) + 1,
        "counterexample": ""
    }

    return result

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(30))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
```

```
    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fd822c45.py", line 101, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fd822c45.py", line 82, in run_trial
    read_once_dim = tropical_dimension(tropical_matrix(read_once_bp))
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fd822c45.py", line 44, in tropical_dime
    if all(abs(matrix[j][i]) == float('inf') for j in range(n)):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fd822c45.py", line 44, in <genexpr>
    if all(abs(matrix[j][i]) == float('inf') for j in range(n)):
               ~~~~~~~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with IndexError during tropical dimension calculation, preventing validation of conjecture | next: Debug tropical_matrix implementation for read-once BPs and re-run tests

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	112565
2	propose	ollama_remote	qwen3:8b	0	0	124423

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	propose	ollama_remote	qwen3:8b	0	0	112698
4	propose	ollama_remote	qwen3:8b	0	0	108095
5	preregistration	ollama_remote	qwen3:8b	0	0	24520
6	novelty	ollama_remote	qwen3:8b	0	0	20741
7	novelty	ollama_remote	qwen3:8b	0	0	12970
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	37603
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10605
10	judge	ollama_remote	qwen3:8b	0	0	20865

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 585086 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/d35dbb9412cf.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d35dbb9412cf.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d35dbb9412cf.tar.gz` (if generated)